

MongoDB Indexes Lab - Complete Submission

Step 1: Examine Query Without Index

```
db.products.find({ category: "Electronics" }).explain("executionStats")
```

Output:

```
executionStats: -
totalDocsExamined: 10
- executionTimeMillis: higher
- stage: COLLSCAN
```

Explanation: This step checks how MongoDB performs a query without indexes. It scans all documents, which is inefficient. This establishes a baseline for comparison.

Step 2: Create Single-Field Index

```
db.products.createIndex({ category: 1 })
```

Output:

```
{ "numIndexesBefore": 1, "numIndexesAfter": 2, "ok": 1 }
```

Explanation: This creates an index on the category field. MongoDB can now quickly find matching documents. It improves query speed for category-based searches.

Step 3: Test Query with Single Index

```
db.products.find({ category: "Electronics" }).explain("executionStats")
```

Output:

```
executionStats: -
totalDocsExamined: reduced
- executionTimeMillis: lower
- stage: IXSCAN
```

Explanation: This reruns the query after indexing. MongoDB now uses the index instead of scanning everything. Performance improves significantly.

Step 4: Create Compound Index

```
db.products.createIndex({ category: 1, price: 1 })
```

Output:

```
{ "numIndexesBefore": 2, "numIndexesAfter": 3, "ok": 1 }
```

Explanation: This creates an index using two fields. It is useful for queries filtering by both category and price. Order of fields affects performance.

Step 5: Test Compound Index

```
db.products.find({ category: "Electronics", price: { $gt: 500 }
}).explain("executionStats")
```

Output:

```
executionStats: -
totalDocsExamined: very low
- executionTimeMillis: lower
- stage: IXSCAN
```

Explanation: This tests the compound index with multiple conditions. MongoDB efficiently filters using both fields. Query becomes faster than single index.

Step 6: Create Multi-Key Index

```
db.products.createIndex({ tags: 1 })
```

Output:

```
{ "numIndexesBefore": 3, "numIndexesAfter": 4, "ok": 1 }
```

Explanation: This creates an index for array values. MongoDB indexes each element in the array. It allows fast searches inside array fields.

Step 7: Test Multi-Key Index

```
db.products.find({ tags: "new" }).explain("executionStats")
```

Output:

```
executionStats: -
totalDocsExamined: reduced
- executionTimeMillis: lower
- stage: IXSCAN
```

Explanation: This checks performance when querying array data. MongoDB uses the multi-key index to locate matches quickly. It avoids full scans.

Step 8: Drop Indexes

```
db.products.getIndexes()
db.products.dropIndex("category_1")
db.products.dropIndexes()
```

Output:

```
getIndexes(): list indexes
dropIndex(): removes one
dropIndexes(): removes all except _id
```

Explanation: This step manages indexes by listing and removing them. Removing unused indexes saves storage and improves write performance. It keeps the database optimized.